

LevelUp English

Mapeamento de Telas (Figma) → Backend · Documento de Planejamento de Desenvolvimento · TCC

1. Visão Geral da Arquitetura

API REST em **Node.js + Express + MongoDB (Mongoose)**, organizada em camadas bem definidas:

Route → **Controller** → **Service** → **Repository** → **Model**

- **Autenticação:** JWT com *access token* + *refresh token*, middleware `AuthMiddleware` e controle de permissão `AuthPermission`.
- **Validação:** schemas dedicados por domínio em `src/schemas/`.
- **Tratamento de erros:** `asyncWrapper` + `errorHandler` + `CustomError` / `CommonResponse` padronizando respostas.
- **Documentação:** Swagger registrado em `src/docs/registry.js`.
- **Infra:** Docker + docker-compose, logs com Winston (rotação diária), seeds com dados fake.

2. Modelos de Dados Existentes

MODEL	CAMPOS PRINCIPAIS
User	<code>name</code> , <code>email</code> , <code>password</code> , <code>xp</code> , <code>level</code> , <code>class</code> , <code>role</code> (student/teacher/admin), <code>mission_progress[]</code> (mission_id, done, score), <code>streak</code> , <code>badges[]</code> , <code>active</code> + tokens
Class	<code>name</code> , <code>active</code> , <code>teacher</code> , <code>students[]</code> , <code>missions[]</code>
Mission	<code>title</code> , <code>description</code> , <code>type</code> (quiz/vocabulary/audio), <code>questions[]</code> (embedded: question, options a–d, correct_answer), <code>content</code> , <code>content_url</code> , <code>xp_reward</code> , <code>class_id</code> , <code>createdBy</code>
Ranking	<code>type</code> (global/class), <code>class</code> , <code>entries[]</code> (user, xp, level)
Attitude	<code>name</code> , <code>description</code> , <code>xp_value</code> , <code>type</code> (positive/negative), <code>createdBy</code>
AttitudeLog	<code>student</code> , <code>attitude</code> , <code>teacher</code> , <code>xp_applied</code> , <code>applied_at</code>

3. Rotas Já Implementadas

Autenticação — `authRoutes.js`

MÉTODO	ROTA	DESCRIÇÃO
POST	<code>/auth/login</code>	Login, retorna tokens
POST	<code>/auth/register-student</code>	Cadastro de aluno (autenticado)

POST	<code>/auth/refresh</code>	Renova access token
POST	<code>/auth/forgot-password</code>	Envia código de recuperação por e-mail
POST	<code>/auth/reset-password</code>	Redefine senha com o código
PATCH	<code>/auth/change-password</code>	Troca de senha autenticada
POST	<code>/auth/logout</code>	Encerra sessão
POST	<code>/auth/revoke/:userId</code>	Revoga tokens de um usuário

Usuários, Turmas e Missões

MÉTODO	ROTA	DESCRIÇÃO
GET	<code>/users</code> · <code>/users/:id</code>	Listagem paginada / detalhe
POST	<code>/users</code>	Criação
PATCH	<code>/users/:id</code>	Atualização
DELETE	<code>/users/:id</code>	Remoção
GET	<code>/classes</code> · <code>/classes/:id</code>	Listagem paginada / detalhe
POST	<code>/classes</code>	Criação de turma
PATCH	<code>/classes/:id</code>	Atualização
DELETE	<code>/classes/:id</code>	Remoção
GET	<code>/missions</code> · <code>/missions/:id</code>	Listagem paginada / detalhe
POST	<code>/missions</code>	Criação (quiz / vocabulary / audio)
PATCH	<code>/missions/:id</code>	Atualização
DELETE	<code>/missions/:id</code>	Remoção

Todas as rotas acima (exceto login, refresh, forgot/reset password) exigem `authMiddleware`.

4. Mapeamento: Telas do Figma → Backend

4.1 Área do Aluno

TELA	STATUS	BACKEND NECESSÁRIO
Landpage / Dashboard boas-vindas, XP, nível, streak, missões recomendadas	Parcial	Existe <code>GET /users/:id</code> cru. Falta um endpoint agregado (ex.: <code>GET /dashboard/student</code>) devolvendo XP total, nível, streak, contadores e missões recomendadas em uma única chamada.
Missões Disponíveis cards com tipo, XP, botão iniciar	Parcial	<code>GET /missions</code> existe. Falta filtrar automaticamente pela turma do aluno logado e enriquecer com o status de <code>mission_progress</code> (pendente/concluída/pontuação).
Executar Missão player de áudio/vídeo, envio de resposta	Falta	Rota de submissão: <code>POST /missions/:id/submit</code> — corrige respostas, calcula score, credita <code>xp_reward</code> , atualiza <code>mission_progress</code> , nível e streak.
Ranking Global	Falta	Model <code>Ranking</code> existe, mas sem controller, service, repository ou rotas. Criar <code>GET /rankings?type=global</code> e <code>GET /rankings?type=class&class=:id</code> .
Perfil do Aluno avatar, nível, barra de XP, badges, contadores	Parcial	Base vem de <code>GET /users/:id</code> . Falta agregação: missões concluídas, badges conquistadas, posição no ranking.

4.2 Área do Professor

TELA	STATUS	BACKEND NECESSÁRIO
Painel de Controle busca de alunos, cards da turma	Parcial	Listagem de alunos por turma via <code>GET /users</code> com filtro por <code>class</code> . Verificar suporte no <code>UserFilterBuild</code> .
Popup "Aplicar Atitudes"	Falta	Módulo <code>Attitude</code> inteiro ausente. Necessário <code>GET /attitudes</code> (listar) e <code>POST /attitude-logs</code> (aplicar a um aluno, somando/subtraindo XP e gravando o log).
Ajustar Saldo adicionar / remover XP	Falta	Ajuste manual de XP: <code>PATCH /users/:id/xp</code> ou reaproveitar o fluxo de <code>AttitudeLog</code> com atitude avulsa. Deve sempre gerar registro auditável.
Missões (prof) e popups de criar/editar (quiz, vocabulário, áudio)	Pronto	CRUD de <code>/missions</code> cobre os três tipos — o model já suporta <code>questions[]</code> , <code>content</code> e <code>content_url</code> .
Perfil do Professor contadores de turmas, missões, alunos	Falta	Endpoint agregado <code>GET /dashboard/teacher</code> com as métricas exibidas nos cards.

4.3 Área do Administrador

TELA	STATUS	BACKEND NECESSÁRIO
Geral / Admin Central resumo geral, status do sistema	Falta	Endpoint agregado <code>GET /dashboard/admin</code> (totais de alunos, professores, turmas, missões) + healthcheck do sistema.
Turmas + popup "Nova Turma"	Pronto	CRUD de <code>/classes</code> atende. Card exibe contagem de alunos — pode vir do tamanho de <code>students[]</code> .
Gerenciar Alunos + "Criar Novo Aluno"	Pronto	<code>POST /auth/register-student</code> + CRUD de <code>/users</code> . Busca por nome depende do filtro em <code>UserFilterBuild</code> .
Professores + "Novo Professor"	Parcial	Existe apenas <code>register-student</code> . Criar <code>POST /auth/register-teacher</code> ou generalizar a rota de registro aceitando <code>role</code> com permissão de admin.
Atitudes de Aula + popups nova/editar	Falta	CRUD completo de <code>/attitudes</code> — nenhuma camada existe além do model.
Perfil do Admin	Pronto	Atendido por <code>GET /users/:id</code> .

5. Lacunas Consolidadas

1. **Módulo Attitude** — controller, service, repository, schema, rotas e documentação Swagger. Bloqueia 4 telas.
2. **Módulo AttitudeLog** — aplicação de atitude com efeito colateral no XP do aluno e registro auditável.
3. **Módulo Ranking** — leitura calculada a partir do XP dos usuários (global e por turma).
4. **Submissão de missão** — correção, pontuação e ganho de XP.
5. **Registro de professor** — rota separada ou generalização do registro por `role`.
6. **Endpoints agregados de dashboard** — aluno, professor e admin.
7. **Regra de nível e streak** — lógica central de progressão (XP → nível), hoje sem implementação.

6. Roteiro de Desenvolvimento Sugerido

Etapa 1 — Módulo Attitude (maior desbloqueio de telas)

- Criar `AttitudeRepository`, `AttitudeService`, `AttitudeController`, `AttitudeSchema` e `attitudeRoutes` seguindo o mesmo padrão de `Class`.
- Registrar rotas em `src/routes/index.js` e permissões em `AuthPermission`.
- Documentar no Swagger (`src/docs/registry.js`).

Etapa 2 — Aplicação de Atitude (AttitudeLog)

- `POST /attitude-logs` : valida aluno e atitude, soma `xp_value` ao `User.xp`, recalcula nível e grava o log.
- Centralizar o cálculo de nível num helper (ex.: `src/utils/LevelHelper.js`) para reuso na submissão de missão.
- `GET /attitude-logs` com filtro por aluno/professor/período (histórico).

Etapa 3 — Progressão do Aluno

- `POST /missions/:id/submit` — corrige, pontua, credita XP, atualiza `mission_progress` e `streak`.
- Impedir dupla contagem de XP para missão já concluída.

Etapa 4 — Ranking

- `GET /rankings` global e por turma, via `aggregate` ordenando por XP.
- Avaliar se o model `Ranking` vale como cache/snapshot ou se o cálculo em tempo real é suficiente para o escopo do TCC.

Etapa 5 — Dashboards e Ajustes Finais

- Endpoints agregados para aluno, professor e admin.
- Rota de registro de professor.
- Ajuste manual de saldo de XP com auditoria.

- Revisão de permissões por `role` em todas as rotas.

Observação sobre testes: o plano de teste geral já está versionado em `documentacao/testes/planoTeste.md` . Recomenda-se estender esse plano a cada etapa concluída, mantendo a rastreabilidade entre requisito, rota e caso de teste — item bem valorizado na avaliação de TCC.